

1. Define CLR & FCL in Dot Net programming.

Ans :-

Common Language Runtime (CLR): The CLR is the core runtime environment in the .NET Framework. It is responsible for managing the execution of code written in .NET-supported languages like C# or VB.NET. Its primary functions include memory management, garbage collection, exception handling, type safety, and thread management. In essence, the CLR ensures the efficient, secure, and consistent execution of applications across different platforms and programming languages.

Framework Class Library (FCL): The FCL is a comprehensive collection of reusable classes, libraries, and interfaces provided by .NET. It offers pre-built functionality for common tasks, such as file handling, database interaction, data structures, and web services, making development faster and easier. The FCL works seamlessly with the CLR, serving as a toolbox for developers to create robust and dynamic applications.

2. What is a MVC application? Explain its components in detail.

Ans :-

MVC application (Model-View-Controller) is a type of application built using the MVC architectural pattern. This pattern separates the application into three interconnected components: **Model**, **View**, and **Controller**. This separation of concerns makes it easier to organize and maintain applications, as each component has a specific responsibility.

Here are the details about the components of an MVC application:

1. Model:

- The Model represents the application's **data and business logic**.
- It handles data operations, such as retrieving data from a database or performing business calculations.
- For example, in a shopping application, the Model might include a Product object that retrieves and stores information in a database and represents the state of products in the store.

2. View:

- The View handles the **presentation layer** and defines what the user sees.
- It is responsible for rendering the UI by using the data provided by the Model.
- Views are typically HTML templates with embedded Razor syntax (C#) in ASP.NET MVC.
- For example, a ProductDetails.cshtml file would display product information on a web page.

3. Controller:

- The Controller acts as a **mediator** between the Model and the View.
- It processes **user input**, interacts with the Model to retrieve or update data, and selects the appropriate View to render.
- Each Controller class contains methods (known as Action Methods) to handle specific user requests.
- For example, the HomeController might have an action method called Index to fetch and display the home page.

3. What is Razor View Engine ?

Ans:-

The **Razor View Engine** is an integral part of ASP.NET MVC that facilitates the creation of dynamic web pages. It is a syntax that blends HTML and server-side code seamlessly, making it easier for developers to write clean and efficient templates. Here's a detailed explanation based on the content of your PDF:

Key Features of Razor View Engine:

- **Dynamic Content Generation:** Razor enables embedding server-side code (like C#) within HTML using the @ symbol. This allows developers to create web pages with dynamic content effortlessly.
- **Clean Syntax:** Unlike traditional ASP.NET Web Forms, Razor offers a lightweight and compact syntax, which is less cluttered and more readable.
- **Seamless Transition Between Code and Markup:** With Razor, switching between HTML and server-side code is smooth, making it highly intuitive for developers.
- **File Extensions:** Razor views are generally stored in files with the .cshtml extension when using C#.

Example of Razor Syntax:

```
@{  
    var date = DateTime.Now.ToShortDateString();  
    var message = "Hello, World!";  
}  
  
<h2>Today's date is: @date</h2>  
  
<h3>@message</h3>
```

This code snippet dynamically renders the current date and a greeting message onto the webpage.

Advantages of Razor View Engine:

- Simplifies dynamic content rendering by integrating C# with HTML.
- Supports inline expressions (e.g., @VariableName) and multi-line code blocks (e.g., @{ ... }).
- Eliminates the need for explicit markers like <%= ... %> used in older engines.

4. Explain the types of views.

Ans:-

1. Standard Views

- **Description:** These are the default views associated with specific controller actions. They display content based on the data passed by the controller.
- **Location:** Found in the ~/Views/<ControllerName>/ folder.
- **Example:** A view named Index.cshtml corresponds to the Index action in the HomeController.

2. Partial Views

- **Description:** These are reusable components of views that can be rendered within other views. They reduce code duplication and simplify the management of UI elements.

- **Usage:** Often used for sections like headers, footers, or repeating elements (e.g., a product card in a product listing page).
- **Location:** Typically found within the same folder as their parent view or in the Shared folder for broader reuse.
- **Example:** `_Product.cshtml` used to display individual product details within a product list.

3. Layout Views

- **Description:** These define a consistent structure for multiple views, such as common headers, navigation bars, and footers.
- **Usage:** Provide a consistent look and feel across the application by including shared UI components.
- **Example:** `_Layout.cshtml` is the main layout file for the application.
- **How It Works:** Other views inherit the layout using the `@Layout` directive.

4. Shared Views

- **Description:** These are views that can be reused across multiple controllers. They often include generic components like error pages or common templates.
- **Location:** Found in the `~/Views/Shared/` folder.
- **Example:** A `Error.cshtml` file in the Shared folder can be used by different controllers to display error information.

5. Strongly-Typed Views

- **Description:** These views are tightly bound to a model type, allowing direct access to model properties with strong type checking.

5. What is Partial View in MVC?

Ans:-

A Partial View in ASP.NET MVC is a reusable view component that is designed to render a portion of a webpage rather than the entire page. It is typically used to modularize the user interface and reduce code duplication. Partial views are similar to user controls in traditional web development.

Key Features of Partial Views:

1. **Reusable UI Components:** Partial views allow you to create sections of a page, such as headers, footers, sidebars, or repeating elements (e.g., product cards), that can be reused across multiple views.
2. **Lightweight:** Unlike standard views, partial views do not include layout pages, making them lighter and more efficient for rendering specific parts of a webpage.
3. **Separation of Concerns:** They help organize code by separating small, self-contained portions of the UI, improving maintainability and readability.

When to Use Partial Views:

- When multiple pages need to share a common UI element (e.g., navigation menu or a user profile section).
- When you want to divide a complex view into manageable chunks.

- When dynamically updating parts of a page using AJAX calls.

Example:

Suppose you want to display a product listing with each product represented by the same structure. You can create a partial view for the product card.

Creating the Partial View (_ProductCard.cshtml):

html

```
@model Product
```

```
<div class="product-card">
```

```
    <h3>@Model.Name</h3>
```

```
    <p>Price: @Model.Price</p>
```

```
</div>
```

Using the Partial View in a Parent View:

html

```
@model IEnumerable<Product>
```

```
@foreach (var product in Model)
```

```
{
```

```
    @Html.Partial("_ProductCard", product)
```

```
}
```

6. Explain what is the difference between View and Partial View?

Ans:-

View

- A **View** represents the complete user interface of a page.

- It **includes layout pages** (`_Layout.cshtml`) by default, which define the structure of the webpage, such as headers, footers, and navigation menus.
- Views are typically used for rendering **entire pages**.
- They include markup tags like `<html>`, `<head>`, `<body>`, etc., for a full HTML structure.
- **Execution flow:** Before rendering a View, the `ViewStart.cshtml` file is executed if it exists.
- Example: A `Details.cshtml` file that renders a full product details page.

Partial View

- A **Partial View** is a reusable fragment of a user interface, often used for rendering specific sections within a page (like headers, footers, or repeating elements).
- It **does not include layout pages** or depend on `ViewStart.cshtml`.
- Partial Views are lightweight and typically used to render **sections of pages** rather than the entire page.
- They don't include complete HTML markup like `<html>` or `<body>`; they focus on the specific UI segment.
- **Execution flow:** Partial Views are embedded within a parent View using methods like `Html.Partial` or `Html.RenderPartial`.
- Example: A `_ProductCard.cshtml` file that renders a single product card for use in a product list.

Summary of Differences:

Aspect	View	Partial View
Purpose	Renders a full page	Renders part of a page
Layout Page	Includes layout page	Does not include layout page
Usage	Used for complete UI	Used for reusable UI components
HTML Structure	Full HTML structure	Partial, focuses on specific segments
Reusability	Less reusable	Highly reusable across Views

7. What are HTML helpers in MVC?

Ans:-

HTML helpers in ASP.NET MVC are methods that assist in creating HTML elements in a view. They simplify the process of building forms, inputs, links, and other HTML elements, making the development process faster and cleaner. Unlike traditional ASP.NET Web Forms controls, HTML helpers are lightweight and do not rely on an event model or ViewState, providing more flexibility and control over the markup.

Key Features of HTML Helpers:

1. **Lightweight:** They generate plain HTML and are simple to use, ensuring minimal overhead.
2. **Integration with Models:** They can bind directly to model properties, reducing the need for manual data binding.

3. **Customizable:** You can use the built-in helpers or create your own to meet specific needs.

Commonly Used HTML Helpers:

1. **TextBox Helper (@Html.TextBox):**

- Generates an <input> element of type "text".
- Example:

```
@Html.TextBox("Name", "Enter your name", new { @class = "form-control" })
```

2. **Label Helper (@Html.Label):**

- Generates a <label> element for a form field.
- Example:

```
@Html.Label("Name", "Full Name")
```

3. **TextArea Helper (@Html.TextArea):**

- Generates a <textarea> element for multiline text input.
- Example:

```
@Html.TextArea("Description", new { @class = "form-control", rows = 5 })
```

4. **DropDownList Helper (@Html.DropDownList):**

- Creates a dropdown list.

Example:

```
@Html.DropDownList("Category", new  
SelectList(Model.Categories, "Id", "Name"))
```

5. **CheckBox Helper (@Html.CheckBox):**

- Renders a checkbox input element.

Example:

```
@Html.CheckBox("IsAvailable", true)
```

6. **ActionLink Helper (@Html.ActionLink):**

- Creates a hyperlink to an action method.

Example:

```
@Html.ActionLink("Home", "Index", "Home")
```

Example

```
@using (Html.BeginForm("Submit", "Home",  
FormMethod.Post))
```

```
{
```

```
<div class="form-group">
```

```
    @Html.LabelFor(m => m.Name)
```

```
    @Html.TextBoxFor(m => m.Name, new { @class = "form-  
control" })
```

```
    @Html.ValidationMessageFor(m => m.Name)
```

```
</div>
```

```
<div class="form-group">
```

```
    @Html.LabelFor(m => m.Email)
```

```
    @Html.TextBoxFor(m => m.Email, new { @class = "form-  
control" })
```

```
@Html.ValidationMessageFor(m => m.Email)

</div>

<button type="submit" class="btn btn-
primary">Submit</button>
```

23. What are HTML helpers? Explain any five HTML helpers.

Ans:-

1. TextBox Helper (@Html.TextBox):

- Generates an <input> element of type "text".
- Syntax:

```
@Html.TextBox("Name", "Enter your name", new { @class = "form-
control" })
```

- The first parameter is the field's name, the second sets its value, and the third adds HTML attributes like a class or ID.

2. Label Helper (@Html.Label):

- Creates a <label> element linked to a form field.
- Syntax:

```
@Html.Label("Name", "Full Name")
```

- The first parameter is the field name, and the second is the label text.

3. DropDownList Helper (@Html.DropDownList):

- Generates a dropdown list (<select> element).

- Syntax:

```
@Html.DropDownList("Category", new SelectList(Model.Categories, "Id", "Name"), "Select a category")
```

- The first parameter is the name of the dropdown, the second is the data source, and the third is the default display text.

4. CheckBox Helper (@Html.CheckBox):

- Generates an <input> element of type "checkbox".

Syntax:

```
@Html.CheckBox("IsAvailable", true, new { @class = "form-check-input" })
```

- The first parameter is the field name, the second sets its checked state (true/false), and the third adds HTML attributes.

5. ActionLink Helper (@Html.ActionLink):

- Creates an anchor link (<a> element) to a specific action method.
- Syntax:

```
@Html.ActionLink("Home", "Index", "Home")
```

- The first parameter is the link text, the second is the action name, and the third is the controller.

24. In Razor files how do you define multi line blocks and inline blocks ? explain with syntax.

Ans:-

1. Multi-Line Code Blocks

- A **multi-line block** contains multiple lines of C# code enclosed in curly braces { }.
- It starts with the @ symbol and allows you to write more complex logic.

Syntax:

razor

@{

// Multi-line block

var date = DateTime.Now.ToShortDateString();

var message = "Hello, Razor!";

int number = 42;

}

<h2>Today's Date: @date</h2>

<h3>Message: @message</h3>

<p>Number: @number</p>

Explanation:

- The block starts with @{ and ends with }.
- Each statement ends with a semicolon ;.
- You can declare variables, write conditional logic, or perform calculations inside the block.

2. Inline Expressions

- An **inline expression** is a single line of server-side code that starts with the @ symbol and directly outputs the result to the HTML.

Syntax:

razor

<p>The current year is @DateTime.Now.Year</p>

Explanation:

- Inline expressions are concise and do not require curly braces { }.
- They are primarily used for displaying simple values or properties directly in the HTML.